Секреты: что было исправлено

Найдено при аудите

Проблема	Где	Риск
Пароль приложения Gmail зашит в код как значение по умолчанию (`SMTP_PASSWORD`)	`server.py`	полный доступ к почтовому ящику у любого, кто видел репозиторий
Админ-код `ADMIN123` захардкожен	`server.py`, `/api/auth/admin-login`	**эскалация привилегий**: любой зарегистрированный пользователь становится администратором
CORS: `allow_origins=["*"]` вместе с `allow_credentials=True` и `allow_origin_regex=".*"`	`server.py`	межсайтовые запросы с cookie-сессией
Keep-alive поток запускался при импорте модуля	`server.py`	сетевые вызовы в тестах и CI

Что сделано

1. Учётные данные SMTP читаются только из окружения; при отсутствии конфигурации `email_configured()` возвращает `False` и письмо не отправляется (приложение работает).
2. Админ-код берётся из `ADMIN_CODE`; при пустом значении маршрут отвечает **503** («не настроен»), а не пропускает всех. Сравнение — `secrets.compare_digest` (устойчиво к таймингу).
3. CORS управляется `CORS_ORIGINS`, по умолчанию — только тот же origin; методы и заголовки перечислены явно.
4. Keep-alive включается `KEEP_ALIVE_ENABLED=true` и никогда не запускается при `ENV=test`.
5. Добавлены регрессионные тесты: `tests/test_security.py` падает, если вернуть любое из этих умолчаний.

Что обязан сделать владелец

- **Отозвать пароль приложения Gmail** (`xtsd…`): он был в открытом репозитории. Gmail → Безопасность → Пароли приложений → удалить.
- Задать `ADMIN_CODE` — длинный случайный код, а не `ADMIN123`.
- Если приложение уже работало публично, проверить, не повышал ли кто-то себе роль: `SELECT id, phone, role FROM users WHERE role = 'admin';`

Правила на будущее

1. В коде нет рабочих секретов — только `os.environ.get(...)` с пустым значением по умолчанию.
2. Отсутствие секрета выключает функцию или отвечает 503, но никогда не включает небезопасный режим.
3. `.env` не коммитится (`.gitignore`), пример — `.env.example` с заглушками.

4. `make doctor` предупреждает, если `ADMIN_CODE` оставлен как в примере.

Очистка истории (2026-10-29)

Полное сканирование истории нашло в старых коммитах токен Telegram-бота (`bot/main.py`, файл позже удалён) и base64-ключ в `DEPLOY_RENDER.md`.

Выполнено `git filter-repo --replace-text`: секретные литералы заменены на `REMOVED_SECRET` во всех коммитах и во всех ветках, изменения отправлены force-push. Проверка на свежем клоне: `gitleaks git` — 0 находок.

Что это меняет и чего не меняет:

1. **Пароль приложения Gmail и токен бота всё равно нужно отозвать** — они были доступны публично; очистка истории не отменяет утечку.
2. Ссылки `refs/pull/*` обновить нельзя (скрытые ссылки, force-push запрещён), поэтому старые объекты остаются доступны по SHA до запроса в GitHub Support.
3. Клоны, сделанные до перезаписи, нужно удалить и клонировать заново.